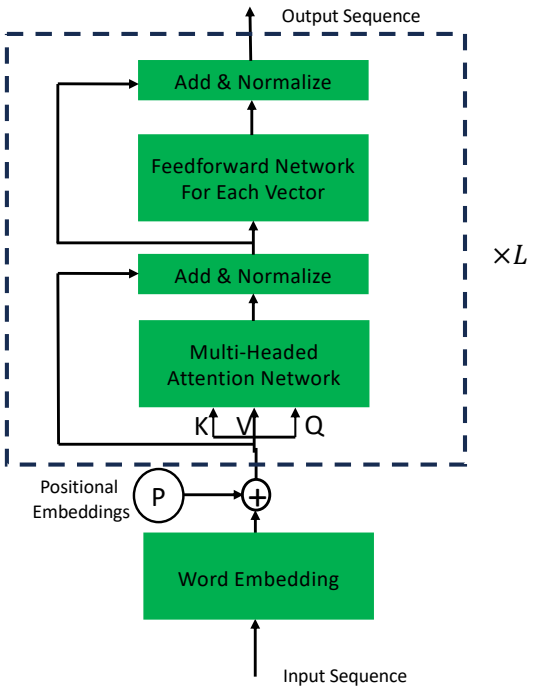
Seeking to Understand Transformer Mechanisms

Lecture 10

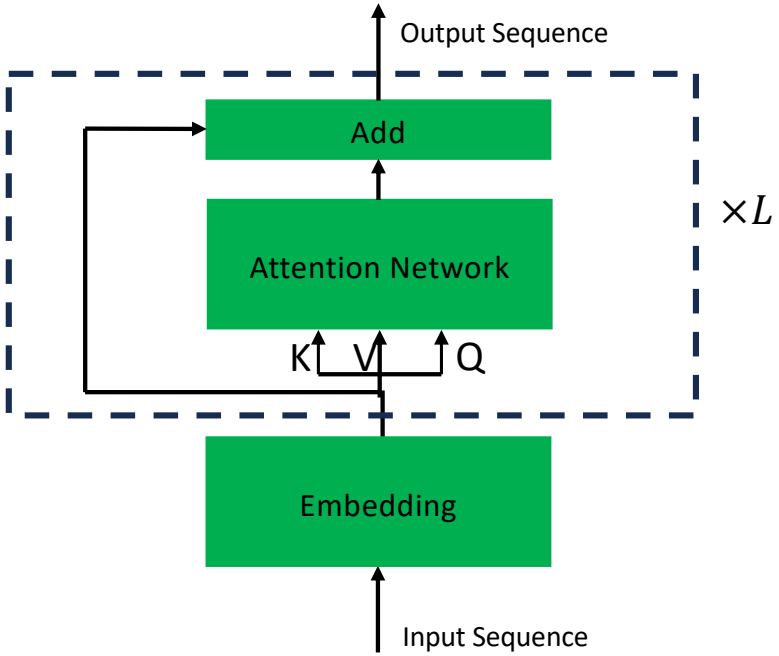
February 10, 2026

Transformer Architecture



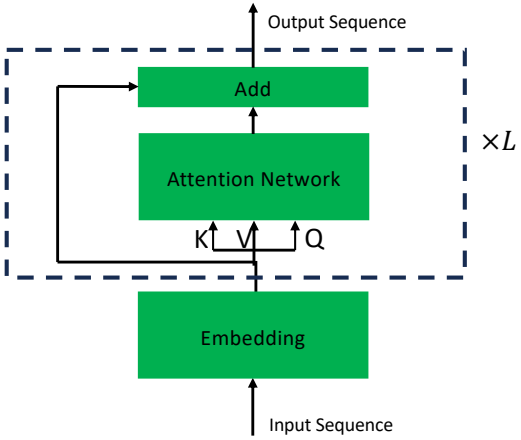
- We have some loose intuition for the attention network and its role
- Skip connections sensible, but role not very clear
- What is the feedforward element doing?

Is Attention All You Need?



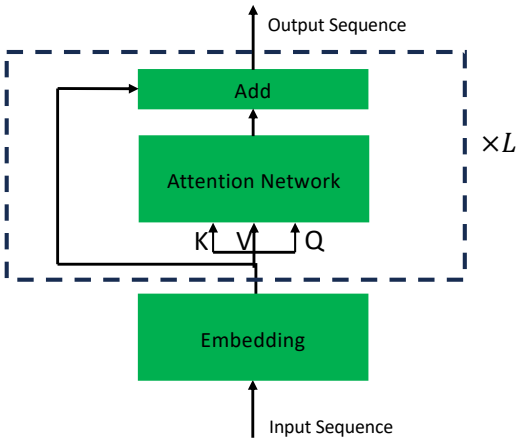
- Let's isolate and try to understand one element of model: Attention plus skip connection
- What does this part of the model do?
- For now also remove positional encoding (will not be needed in problems we will examine)

Encoding at Each Layer



- Assume that we have N inputs, and let $e_i^{(0)} \in \mathbb{R}^d$ represent the vector at position $i = 1, \ldots, N$ input to attention layer $l = 1$
- Simplify by considering only one attention head, with parameters $W_Q \in \mathbb{R}^{d \times d}$, $W_K \in \mathbb{R}^{d \times d}$, $W_V \in \mathbb{R}^{d \times d}$ and $P \in \mathbb{R}^{d \times d}$
- Assume W_Q, W_K, W_V and P are the same at all attention layers (simplification)

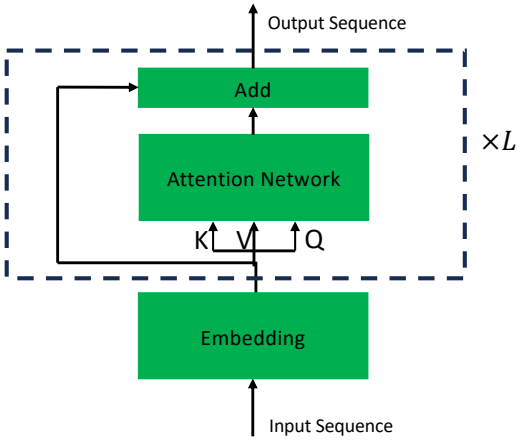
First Attention Layer



- The output at position i from first attention layer, plus skip connection, is

$$
\begin{aligned}
e_i^{(1)} &= e_i^{(0)} + P \sum_{j=1}^{N} (W_V e_j^{(0)}) w_{ij}^{(0)} \\
w_{ij}^{(0)} &= \frac{\exp[\lambda (W_Q e_i^{(0)})^T (W_K e_j^{(0)})]}{\sum_{j'=1}^{N} \exp[\lambda (W_Q e_i^{(0)})^T (W_K e_{j'}^{(0)})]}
\end{aligned}
$$

First Attention Layer



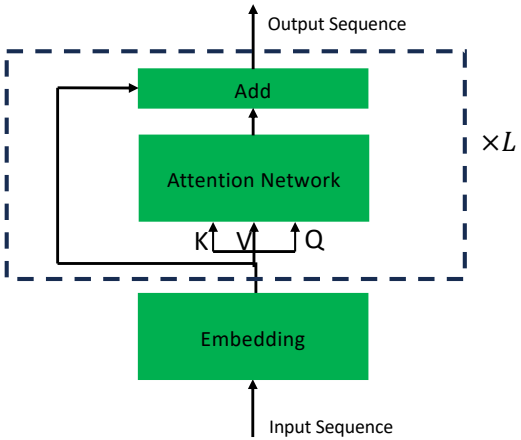
- The output at position i from first attention layer, plus skip connection, is

$$e_i^{(1)} = e_i^{(0)} + P \sum_{j=1}^{N} (W_V e_j^{(0)}) w_{ij}^{(0)}$$

- The weight $w_{ij}^{(0)}$ is a measure of how much "attention" should be placed on position j, when updating vector at position i
- Let's simplify this: Let

$$w_{ij}^{(0)} = (W_Q e_i^{(0)})^T (W_K e_j^{(0)}) = (e_i^{(0)})^T W_Q^T W_K e_j^{(0)} = (e_i^{(0)})^T \Phi e_j^{(0)}$$

Inner-Product-Based Attention

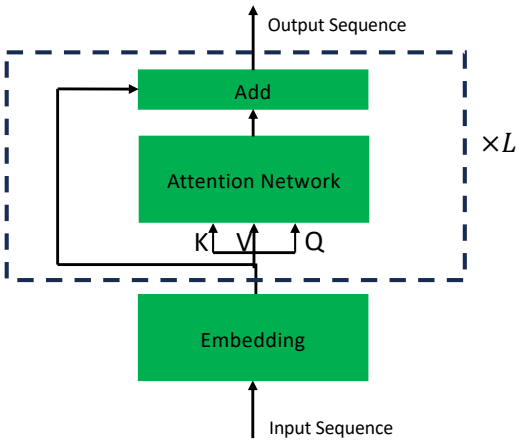


- Further simplify the setup such that Φ is the identity matrix
- The update rule for layer one plus skip connection is now

$$e_i^{(1)} = e_i^{(0)} + P \sum_{j=1}^{N} (W_V e_j^{(0)}) \underbrace{[(e_i^{(0)})^T e_j^{(0)}]}_{\text{linear attention}}$$

- A simplification, but captures most elements of the attention network, plus skip connection

Extend to Multiple Layers



- The update rule for layer $l = 1, \ldots, L$, plus skip connections, is now

$$e_i^{(l)} = e_i^{(l-1)} + P \sum_{j=1}^{N} (W_V e_j^{(l-1)})[(e_i^{(l-1)})^T e_j^{(l-1)}]$$

- Have assumed, for simplicity that W_Q, W_K, W_V and P are the same at all layers (need not be so in general)

Matrix Representation

- The update rule for layer $l = 1, \ldots, L$, plus skip connections, is now

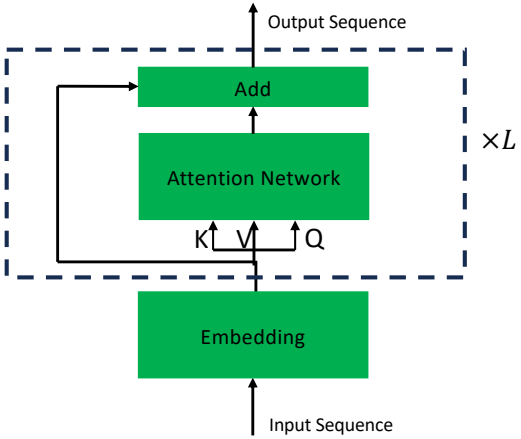
$$e_i^{(l)} = e_i^{(l-1)} + P \sum_{j=1}^{N} (W_V e_j^{(l-1)})[(e_i^{(l-1)})^T e_j^{(l-1)}]$$

- Define the matrix $E_l = (e_1^{(l)}, \ldots, e_N^{(l)}) \in \mathbb{R}^{d \times N}$

- In matrix form, the update rule for all N positions may be expressed

$$E_l = E_{l-1} + P \underbrace{W_V E_{l-1}}_{\text{Values}} \underbrace{[E_{l-1}^T E_{l-1}]}_{\text{Attention}}$$

View The Forward Pass of Attention Layers as Few-Shot Learning



$$E_l = E_{l-1} + P \underbrace{W_V E_{l-1}}_{\text{Values}} \underbrace{[E_{l-1}^T E_{l-1}]}_{\text{Attention}}$$

- Given *context* ("prompt") associated with the N inputs, predict the next sample $N+1$
- The N inputs are viewed as the context, or examples
- Termed few-shot learning: The N samples are contextual examples, and based on them predict $N+1$

Learning to Learn: Few-Shot Learning

- Few-shot learning is a widely studied field in machine learning
- Consider contextual data $\mathcal{C} = \{(x_i, y_i)\}_{i=1,N}$, where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$
- Assume that there is an unknown function $f(x) = y$ responsible for the outcomes: $y_i = f(x_i)$
- Given the context $\mathcal{C}$, infer the latent function $f(x)$, and use the inferred function to predict an outcome $y_{N+1} = f(x_{N+1})$ for a "query" x_{N+1}

Meta Learning

- Few shot learning has also been termed meta learning
- Wish to learn a meta model, that is able to adapt to new context, and make predictions that are context-dependent
- Ideally, once we learn the meta model, its parameters do not change as it subsequently adapts to new context
- Train the meta model such that it is capable of continuing to adapt, without parameter change
- Overall goal: Learning to learn

Learning to Learn: Training Data and Process

- Consider training data composed of M sets of contextual data, $\mathcal{C}_m$, for $m = 1, \ldots, M$, where

$$\mathcal{C}_m = \{(x_i^{(m)}, y_i^{(m)})\}_{i=1,N+1}$$

- Train the meta model to take in context $\{(x_i^{(m)}, y_i^{(m)})\}_{i=1,N}$ and the query $x_{N+1}^{(m)}$
- The meta model should be able to predict $y_{N+1}^{(m)}$ given the context and the query

Learning to Learn: Training Data and Process

- Let the Transformer be represented symbolically as $T_\phi(z)$, where ϕ are the Transformer parameters and z represents the input set of vectors
- For context m, encode $\{(x_i^{(m)}, y_i^{(m)})\}_{i=1,N}$ and the query $x_{N+1}^{(m)}$ as z, and then use the Transformer to predict $y_{N+1}^{(m)}$
- View the Transformer (here only in terms of attention and skip connection) as a meta learner
- Once a Transformer has been trained, can adapt to new prompts, with no change of Transformer parameters

Few Shot Learning With Linear Functions

- Consider contextual data

$$\mathcal{C}_m = (x_1^{(m)}, y_1^{(m)}, \ldots, x_N^{(m)}, y_N^{(m)})$$

with $y_i^{(m)} = f_{\theta_m}(x_i^{(m)})$

- We now focus on linear model, $f_{\theta_m}(x) = w_m^T x$ where $\theta_m = w_m \in \mathbb{R}^d$

- For example, we may assume each $w_m \sim \mathcal{N}(0_d, I_d)$

- Also, may assume all covariates drawn $x_i^{(m)} \sim \mathcal{N}(0_d, I_d)$

Learning to Learn With Linear Functions

- For the mth contextual dataset

$$\mathcal{C}_m = (x_1^{(m)}, y_1^{(m)}, \ldots, x_N^{(m)}, y_N^{(m)})$$

  assume a linear model, $f_{\theta_m}(x) = w_m^T x$ where $w_m \in \mathbb{R}^d$

- The latent w_m is different for each set of contextual data

- Can we train a meta model (the attention model) to take $\mathcal{C}_{M+1}$ as input, along with the query $x_{N+1}^{(M+1)}$, and to predict $y_{N+1}^{(M+1)}$ for an underlying w_{M+1} it has never seen?

- The trained model is trained such that it can perform inference based on a few examples (few-shot learning)

Gradient Descent for Linear Model, Data $\mathcal{C}_m$

- Hypothesis: Maybe the attention-based model can learn to perform gradient descent in its forward pass, based on the observed contextual data

- Cost function:

$$\mathcal{L}(w) = \frac{1}{2}\sum_{i=1}^{N}(y_i - w^T x_i)^2$$

where we have removed subscript m for simplicity

- Gradient descent update:

$$w_{k+1} = w_k + \alpha \sum_{i=1}^{N}(y_i - w_k^T x_i)x_i$$

for gradient-descent step $k \geq 0$

Gradient Descent for Linear Model: Interpretation

- Gradient descent update:

$$w_{k+1} = w_k + \alpha \sum_{i=1}^{N} (y_i - w_k^T x_i) x_i$$

- If $y_i - w_k^T x_i > 0$ then the model weights w_k are putting too little emphasis in the direction of x_i
- Hence, if $y_i - w_k^T x_i > 0$, then add *positive* weighting in direction of x_i: $\alpha(y_i - w_k^T x_i)x_i$ added to w_k when updating
- α is the learning rate
- Similar if $y_i - w_k^T x_i < 0$, then subtract from w_k vector in direction of x_i

Functional Model Update

- Using

$$w_{k+1} = w_k + \alpha \sum_{i=1}^{N} (y_i - w_k^T x_i) x_i$$

the model is updated as

$$\begin{aligned} f_{\theta_{k+1}}(x_j) &= w_k^T x_j + \alpha \sum_{i=1}^{N} (y_i - w_k^T x_i) x_j^T x_i \\ &= f_{\theta_k}(x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] x_j^T x_i \end{aligned}$$

Functional Model Update & Linear Attention

- The update rule for GD with linear model is

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] x_j^T x_i$$

- Recall the update rule for linear attention:

$$e_{j,k+1} = e_{j,k} + P[W_V E][E^T e_{j,k}]$$

- Can we construct P, W_V, Φ and the form of e_i such that we achieve an equivalence?

Few Shot Learning with Linear Attention Transformer

- Given context $\mathcal{C} = (x_1, y_1, \ldots, x_N, y_N)$ and a query x_{N+1}

- GD update rule:

$$\begin{aligned} f_{\theta_{k+1}}(x_j) &= f_{\theta_k}(x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] x_i^T x_j \\ &= f_{\theta_k}(x_j) + \underbrace{\alpha \tilde{Y}_k X^T x_j}_{\Delta f_{\theta_{k+1}}(x_j)} \end{aligned}$$

where $X = (x_1, \ldots, x_N)$, $\tilde{Y}_k = (y_1 - f_{\theta_k}(x_1), \ldots, y_N - f_{\theta_k}(x_N))$

- Build up the function iteratively:

$$f_{\theta_k}(x_j) = f_{\theta_0}(x_j) + \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j)$$

Few Shot Learning with Linear Attention Transformer

- Assume $w_0 = 0_d$, so $f_{\theta_0}(x_j) = 0$; GD update rule:

$$
\begin{aligned}
f_{\theta_{k+1}}(x_j) &= \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) + \underbrace{\alpha \tilde{Y}_k X^T x_j}_{\Delta f_{\theta_{k+1}}(x_j)} \\
X &= (x_1, \ldots, x_N) \\
\tilde{Y}_k &= (y_1 - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_1), \ldots, y_N - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_N))
\end{aligned}
$$

- The update $\Delta f_{\theta_{k+1}}(x_j)$ seems to align perfectly with the form of linear attention

- Values at layer k: $\tilde{Y}_k$; Keys: X; jth Query: x_j

Sequential Buildup of Function

- Output of attention at layer $k+1$:

$$
\begin{aligned}
f_{\theta_{k+1}}(x_j) &= \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) + \underbrace{\alpha \tilde{Y}_k X^T x_j}_{\Delta f_{\theta_{k+1}}(x_j)} \\
X &= (x_1, \ldots, x_N) \\
\tilde{Y}_k &= (y_1 - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_1), \ldots, y_N - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_N))
\end{aligned}
$$

- Build up $f_\theta(x_j)$ layer by layer
- Input to layer $k+1$: $\tilde{Y}_k$ and X
- Differential output at layer $k+1$: $\Delta f_{\theta_{k+1}}(x_j)$, $\forall j = 1, \ldots, N+1$

Sequential Buildup of Function

- Output of attention at layer $k+1$:

$$f_{\theta_{k+1}}(x_j) \quad = \quad \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) + \underbrace{\alpha \tilde{Y}_k X^T x_j}_{\Delta f_{\theta_{k+1}}(x_j)}$$

- *Differential* output at layer $k+1$: $\Delta f_{\theta_{k+1}}(x_j)$, $\forall j = 1, \ldots, N+1$
- *Do not* output $f_{\theta_{k+1}}(x_j)$, rather output the *change*, $\Delta f_{\theta_{k+1}}(x_j)$
- Via skip connection, we add $\Delta f_{\theta_{k+1}}(x_j)$ to the input $f_{\theta_k}(x_j)$, to get the total output $f_{\theta_{k+1}}(x_j)$

Detailed Construction

- Assume that the parameters are initialized $w_0 = 0_d$ and employ the following encoding as input to layer $k = 1$ attention:

$$\underbrace{\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}}_{e_{1,0}} \underbrace{\begin{pmatrix} x_2 \\ y_2 \end{pmatrix}}_{e_{2,0}} \quad \cdots \quad \begin{pmatrix} x_N \\ y_N \end{pmatrix} \underbrace{\begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}}_{e_{N+1,0}}$$

- The update rule for linear attention is

$$e_{j,k+1} = e_{j,k} + P[W_V E][E^T \Phi e_{j,k}]$$

- Design Φ, W_V and P as

$$\Phi = \begin{pmatrix} I_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix}, \quad W_V = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 1 \end{pmatrix}, \quad P = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & -\alpha \end{pmatrix}$$

Summary of Update Process via Linear Attention (1/3)

- Input to linear attention at layer k, position $j = 1, \ldots, N+1$:

$$e_{j,k} = \begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}$$

  with y_{N+1} set to zero

- Keys ($j = 1, \ldots, N$) and queries ($j = 1, \ldots, N+1$):

$$W_Q e_{j,k} = W_K e_{j,k} = \begin{pmatrix} x_j \\ 0 \end{pmatrix}$$

- Values ($j = 1, \ldots, N$)

$$W_V e_{j,k} = \begin{pmatrix} 0_{d \times 1} \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}$$

Summary of Update Process via Linear Attention (2/3)

- Input to linear attention at layer k, position $j = 1, \ldots, N+1$:

$$e_{j,k} = \begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}$$

- Projections:

$$W_Q e_{j,k} = W_K e_{j,k} = \begin{pmatrix} x_j \\ 0 \end{pmatrix}, \quad W_V e_{j,k} = \begin{pmatrix} 0_{d\times 1} \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}$$

- Linear attention, $P \sum_{i=1}^{N} W_V e_{i,k} [W_K e_{i,k}]^T [W_Q e_{j,k}]$, yields:

$$-\alpha \sum_{i=1}^{N} \begin{pmatrix} 0_{d\times 1} \\ y_i - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_i) \end{pmatrix} x_i^T x_j = \begin{pmatrix} 0_{d\times 1} \\ -\Delta f_{\theta_{k+1}}(x_j) \end{pmatrix}$$

Summary of Update Process via Linear Attention (3/3)

$$\underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{j\text{th input to layer } k+1 \text{ attention}} \implies \underbrace{\begin{pmatrix} 0_{d\times 1} \\ -\Delta f_{\theta_{k+1}}(x_j) \end{pmatrix}}_{j\text{th output from layer } k+1 \text{ attention}}$$

$$\implies \underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k+1} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{\text{skip connection: add input and output}}$$

- The output for the query x_{N+1} is

$$\begin{pmatrix} x_{N+1} \\ -\sum_{k'=1}^{k+1} \Delta f_{\theta_{k'}}(x_{N+1}) \end{pmatrix}$$

- Inner product with $(0_d, -1)^T$ after final layer of Transformer

Summary

- A Transformer with linear attention, with properly designed parameters, can perform gradient descent in its forward pass for linear functions
- The GD is based on a few shots of contextual data, plus a query x_{N+1}
- Important: The setup does not explicitly update the model parameters θ of $f_\theta(x)$
- It updates the functional approximation, layer by layer, in a manner consistent with GD
- Question 1: Are the above designed parameters ϕ for the Transformer optimal?
- Question 2: Linear attention Transformer only works for linear functions. How do we extend for nonlinear functions?

Summary (1/3)

$$\text{Softmax attention: } e_i \quad \rightarrow \quad e_i + \sum_{h=1}^{H} P^{(h)}[W_V^{(h)} E] \cdot \text{softmax}[E^T \Phi^{(h)} e_i]$$

$$\text{Linear attention: } e_i \quad \rightarrow \quad e_i + \sum_{h=1}^{H} P^{(h)}[W_V^{(h)} E] \cdot [E^T \Phi^{(h)} e_i]$$

- Have demonstrated that for linear data models $f(x) = w^T x$ we can design P, W_V and Φ to implement gradient descent for model weights w with linear-attention Transformer
- Each layer of Transformer implements one step of GD
- Given new contextual data $\mathcal{C} = (x_1, w^T x_1, \ldots, x_N, w^T x_N, x_{N+1})$ linear Transformer can predict $w^T x_{N+1}$ on its forward pass

Summary (2/3)

$$\underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{j\text{th input to layer } k+1 \text{ attention}} \implies \underbrace{\begin{pmatrix} 0_{d\times 1} \\ -\Delta f_{\theta_{k+1}}(x_j) \end{pmatrix}}_{j\text{th output from layer } k+1 \text{ attention}}$$

$$\implies \underbrace{\begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k+1} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}}_{\text{skip connection: add input and output}}$$

- Model does not explicitly update the model parameters w_k
- Updates the function $f_{\theta_k}(x_i) = f_{\theta_0}(x_i) + \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_i)$

Summary (3/3)

- Our hand-crafted design of linear Transformer parameters guided by gradient descent (GD)
- If we train the linear Transformer parameters based on multiple contextual examples $\{\mathcal{C}_m\}_{m=1,M}$ it can do better than GD: *preconditioned* GD
- Accounts for statistics of data and model parameters, $p(X)$ and $p(W)$
- Big assumption: linear models, $f(x) = w^T x$
- Now generalize the idea of linear models: kernel machines

Map to Feature Space

- Map $x \in \mathbb{R}^d$ to features $\phi(x)$
- Dimension of output from $\phi(x)$ is d', which in general different than d
- Dimension d' could even be infinite
- Perform linear regression in the space of features $\phi(x)$

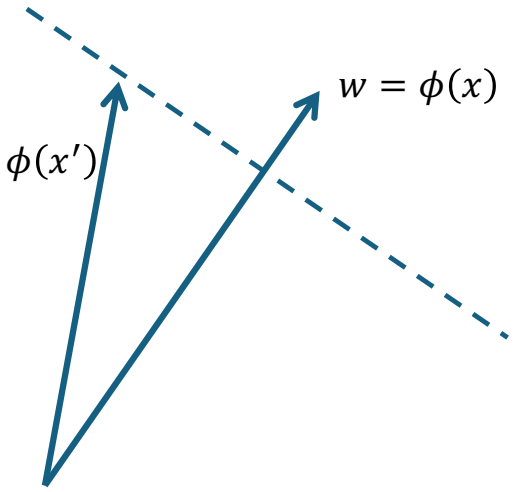
Properties

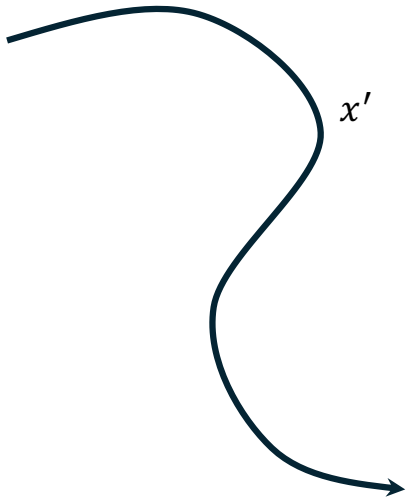
$$f_\theta(x) = w^T \phi(x) + b \quad , \qquad \theta = (w, b)$$

- All $\phi(x)$ along a line perpendicular to w yield the same value for $f_\theta(x)$
- But, if we look at the values of x that manifest $\phi(x)$ along such a perpendicular, typically nonlinear
- In the space of x, surfaces that yield constant functional values are nonlinear
- Key idea: Build nonlinear function $f_\theta(x)$ via nonlinear $x \to \phi(x)$, followed by linear $\phi(x) \to f_\theta(x) = w^T \phi(x) + b$

Nonlinear Features $\phi(x)$ & Linear Model $w^T\phi(x)$

$\phi(x')$ that yields constant $f_\theta(x') = w^T\phi(x') + b$





Values of x' that place $\phi(x')$ on dashed line

Inner Products in Feature Space

- The inner product between $\phi(x_i)$ and $\phi(x_j)$ is
$$\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$
- The expression $\kappa(x_i, x_j)$ is called a Mercer kernel
- We may define a vector in feature space as
$$w = \sum_{m=1}^{M} \alpha_m \phi(x_m)$$
where $\alpha_m \in \mathbb{R}$ and $x_m \in \mathbb{R}^d$
- The function $f(x) = \phi(x)^T w$ may be expressed
$$f_\theta(x) = \sum_{m=1}^{M} \alpha_m \kappa(x, x_m)$$

Kernel Machines

- Assume that the underlying function $f(x)$ responsible for the data is nonlinear
- Select a kernel type $\kappa(x_i, x_j)$
- Seek to fit data to learn a model of the form

$$f_\theta(x) = \sum_{m=1}^{M} \alpha_m \kappa(x, x_m) + b$$

- Do not explicitly define feature mapping $\phi(x)$, work directly with the corresponding kernel

Example Mercer Kernels

- Linear: $\kappa(x_i, x_j) = x_i^T x_j$
- Radial basis function (RBF): $\kappa(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|_2^2}{\sigma^2}\right)$
- Laplacian: $\kappa(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|_1}{\sigma^2}\right)$
- Sigmoid: $\kappa(x_i, x_j) = \tanh(\gamma x_i^T x_j + c)$
- Polynomial: $\kappa(x_i, x_j) = (\gamma x_i^T x_j + c)^d$
- Cosine: $\kappa(x_i, x_j) = \frac{x_i^T x_j}{\|x_i\|_2 \|x_j\|_2}$
- Chi-squared: $\kappa(x_i, x_j) = \exp\left(-\gamma \sum_{k=1}^{d} \frac{(x_i(k) - x_j(k))^2}{x_i(k) + x_j(k)}\right)$

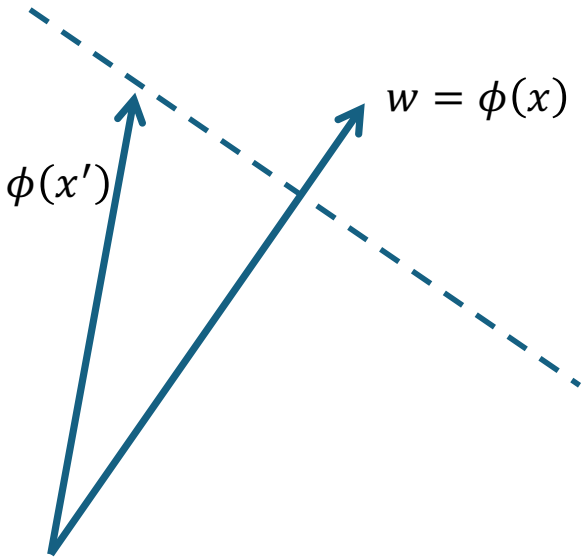
RBF Kernel as an Example

$$\kappa(x', x) = \exp\left(\frac{-\|x - x'\|_2^2}{\sigma^2}\right)$$

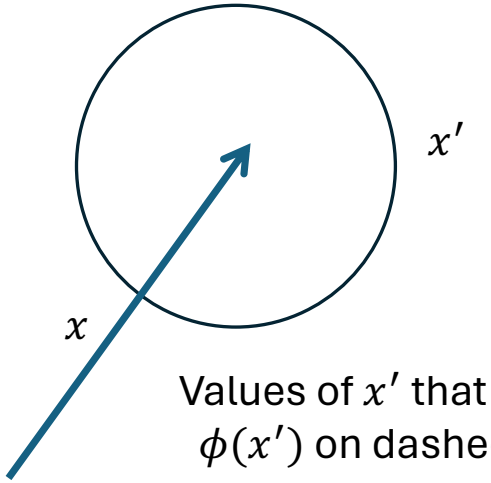
- There is a nonlinear mapping $\phi(x)$ for which $\kappa(x', x) = \phi(x)^T \phi(x')$
- $\phi(x')$ on a hyper-plane perpendicular to $w = \phi(x)$ correspond to positions x' in covariate space on a hypersphere

Radial Basis Function (RBF) Kernel

$\phi(x')$ that yields constant
$f_\theta(x') = w^T\phi(x') + b = \kappa(x', x) + b$



$$\kappa(x', x) = \exp[\frac{-\|x' - x\|_2^2}{\sigma^2}]$$



Values of x' that place $\phi(x')$ on dashed line

RKHS

- The function $\phi(x)$ outputs a d' dimensional vector
- We take inner products in this feature space, in what is termed a Hilbert space
- Inner products in this Hilbert space yield kernel evaluations in the covariate space x
- This is a well studied domain of machine learning, called modeling with a reproducing kernel Hilbert space (RKHS)
- Technical reasons for why termed "reproducing"

Gradient Descent for Linear Model in $\phi(x)$

- Cost function:

$$\mathcal{L}(w) = \frac{1}{2}\sum_{i=1}^{N}[y_i - w^T\phi(x_i)]^2$$

- Gradient descent update:

$$w_{k+1} = w_k + \alpha\sum_{i=1}^{N}[y_i - w_k^T\phi(x_i)]\phi(x_i)$$

- Corresponding function

$$\begin{aligned} f_{w_{k+1}}(x_j) &= w_{k+1}^T\phi(x_j) \\ &= w_k^T\phi(x_j) + \alpha\sum_{i=1}^{N}[y_i - w_k^T\phi(x_i)]\kappa(x_i, x_j) \\ &= f_{w_k}(x_j) + \alpha\sum_{i=1}^{N}[y_i - f_{w_k}(x_i)]\kappa(x_i, x_j) \end{aligned}$$

From Linear Attention to Kernel Attention

- Update rule for nonlinear kernel machine

$$f_{w_{k+1}}(x_j) = f_{w_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{w_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{w_{k+1}}(x_j)}$$

- Like with linear attention, initialize the weight vector w as all zeros vactor, and therefore $f_{w_0}(x_j) = 0$
- With linear attention, we assumed the model bias b was zero
- Can move beyond that assumption now

Gradient Descent for Linear Model in $\phi(x)$

- Cost function:

$$\mathcal{L}(w, b) = \frac{1}{2} \sum_{i=1}^{N} [y_i - w^T \phi(x_i) - b]^2$$

- Gradient descent update:

$$w_{k+1} = w_k + \alpha \sum_{i=1}^{N} [y_i - w_k^T \phi(x_i)] \phi(x_i)$$

$$b_{k+1} = b_k + \alpha \sum_{i=1}^{N} [y_i - w_k^T \phi(x_i)]$$

- Corresponding function

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j) + \alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]$$

Two Attention Heads

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Two attention heads, one with $\kappa(x_i, x_j)$ attention, and the other with constant attention
- Need to return to the form of the Transformer design to see how these two types of attention heads may be manifested in the same model

Transformer Design (1/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Let's assume that we consider the RBF kernel, $\kappa(x_i, x_j) = \exp\left(\frac{-\|x_i - x_j\|_2^2}{\sigma^2}\right)$ to be specific
- Also assume that the input data are encoded is $e_i = (x_i, y_i)^T$ for $i = 1, \ldots, N$ and $e_{N+1} = (x_{N+1}, 0)^T$

Transformer Design (2/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- For attention head 1, can encode input to RBF-kernel attention via

$$\begin{aligned} W_Q^{(1)} = W_K^{(1)} &= \begin{pmatrix} I_d & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix} \\ \kappa(W_K^{(1)} e_i, W_Q^{(1)} e_j) &= \exp\left(\frac{-\| W_K^{(1)} e_i - W_Q^{(1)} e_j \|_2^2}{\sigma^2}\right) \\ &= \exp\left(\frac{-\| x_i - x_j \|_2^2}{\sigma^2}\right) \end{aligned}$$

Transformer Design (3/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- For attention head 2, can encode input to RBF-kernel attention via

$$\begin{aligned} W_Q^{(2)} = W_K^{(2)} &= 0_{(d+1)\times(d+1)} \\ \kappa(W_K^{(2)} e_i, W_Q^{(2)} e_j) &= 1 \end{aligned}$$

Transformer Design (4/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Key and query matrices for two heads:

$$W_Q^{(1)} = W_K^{(1)} = \begin{pmatrix} I_d & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix}, \quad W_Q^{(2)} = W_K^{(2)} = 0_{(d+1)\times(d+1)}$$

Transformer Design (5/5)

$$f_{\theta_{k+1}}(x_j) = f_{\theta_k}(x_j) + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)] \kappa(x_i, x_j)}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 1 attention}} + \underbrace{\alpha \sum_{i=1}^{N} [y_i - f_{\theta_k}(x_i)]}_{\Delta f_{\theta_{k+1}}(x_j),\ \text{type 2 attention}}$$

- Key and query matrices for two heads:

$$W_Q^{(1)} = W_K^{(1)} = \begin{pmatrix} I_d & 0_{d\times 1} \\ 0_{1\times d} & 0 \end{pmatrix}, \quad W_Q^{(2)} = W_K^{(2)} = 0_{(d+1)\times(d+1)}$$

- Value matrices, and output matrices

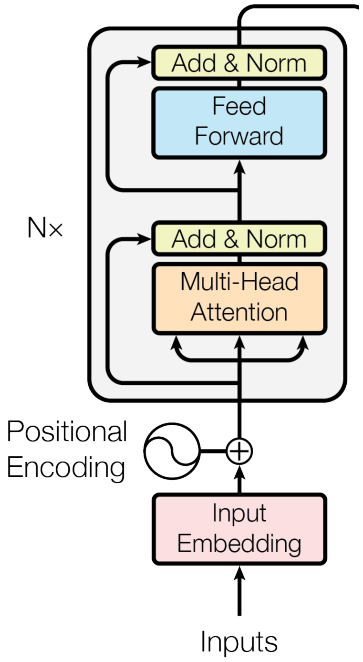
$$W_V^{(1)} = W_V^{(2)} = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & 1 \end{pmatrix}, \quad W_O^{(1)} = W_O^{(2)} = \begin{pmatrix} 0_{d\times d} & 0_{d\times 1} \\ 0_{1\times d} & -\alpha \end{pmatrix}$$

- See the power of nonlinear attention and multiple heads

Softmax Kernel & Multiple Attention Heads

- Above construction with kernel regression gives sense of where the nonlinear attention arises from within Transformer
- However, from the perspective of gradient descent, the kernel needs to be Mercer
- The softmax kernel doesn't fit the Mercer kernel perspective, but there are Mercer kernels that are close to it
- We can see the value of multiple head types within the Transformer
- Role of skip connection in attention also clear; needed to implement GD in forward pass

Review the form of the Transformer



- We have explained potential utility of nonlinear (kernel), multi-headed attention
- Allows in-context inference (e.g., gradient descent) in its forward pass
- What about the FF block?